Creating Maven Ansible Web App:

Step 1: This below Maven command creates a new ansible web application project named **MavenAnsibleWebApp** using the **maven-archetype-webapp** template. It sets the group ID to com.example and runs in non-interactive mode, generating all project files automatically.

mvn archetype:generate -DgroupId=com.example -DartifactId=MavenAnsibleWebApp -DarchetypeArtifactId=maven-archetype-webapp -DinterativeMode=false

```
shweta@shweta-virtualbox:~/Devops/mvn archetype:generate -DgroupId=com.example -DartifactId=MavenAnsibleWebApp -DarchetypeArtifactId=maven-
archetype-webapp -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-----------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -------------------------------- [ pom ] -----------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Interactive mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 10.0 MB/s)
[INFO] Using property: groupId = com.example
[INFO] Using property: artifactId = MavenAnsibleWebApp
Define value for property 'version' 1.0-SNAPSHOT:
[INFO] Using property: package = com.example
Confirm properties configuration:
groupId: com.example
artifactId: MavenAnsibleWebApp
version: 1.0-SNAPSHOT
package: com.example
Y:
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/shweta/Devops
```

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  14.607 s
[INFO] Finished at: 2026-04-15T03:52:36Z
[INFO] ------------------------------------------------------------------------
```

Step 2: Configure Maven Ansible Web App Project (pom.xml)

In this step, we move to the project directory using cd MavenAnsibleWebApp and open the Maven configuration file using gedit pom.xml. Then, we write the required project details, dependencies, WAR packaging settings, and build configuration for creating the web application.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>MavenAnsibleWebApp1</artifactId>

<packaging>war</packaging>

<version>1.0-SNAPSHOT</version>

<name>MavenAnsibleWebApp1</name>

<url>http://maven.apache.org</url>

<dependencies>

<dependency>

<groupId>junit</groupId>

<artifactId>junit</artifactId>

<version>3.8.1</version>

<scope>test</scope>

</dependency>

<!--  Servlet API for Java EE  -->

<dependency>
```

```
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>
<version>4.0.1</version>
<scope>provided</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-war-plugin</artifactId>
<version>3.4.0</version>
<!--  Update to latest stable version  -->
</plugin>
</plugins>
<finalName>MavenAnsibleWebApp1</finalName>
<!--  create a web page with the name MavenAnsibleWebApp, if we want some other name we
can chage it  -->
</build>
</project>
```

Step 3: Create Ansible Configuration Files

In this step, we create a new folder named ansible inside the project directory using mkdir ansible. Then, we move into the folder using cd ansible and create the required Ansible files using gedit hosts.ini and gedit playbook.yml. These files are used to define target hosts and deployment tasks for automation.

```
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp$ mkdir ansible
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp$ ls
ansible  Jenkinsfile  pom.xml  src
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp$ cd ansible
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebbApp/ansible$ gedit hosts.ini
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebbApp/ansible$ ls
hosts.ini
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp/ansible$ gedit playbook.yml
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp/ansible$ ls
hosts.ini  playbook.yml
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp/ansible$
```

gedit playbook.yml

This code is written inside playbook.yml file to automate deployment using Ansible. It targets the web host group, copies the generated WAR file from the Jenkins workspace to the Tomcat webapps folder, and then restarts the Tomcat service so the updated application runs successfully.

```
- name: Deploy to Tomcat
  hosts: web
  become: yes
  tasks:
```

```
- name: Copy WAR file to Tomcat
  copy:
    src: "/var/snap/jenkins/5013/workspace/MavenAnsibleWebApp1-/target/MavenAnsibleWebApp1.war"
    dest: "/opt/tomcat/webapps/MavenAnsibleWebApp1.war"
    remote_src: no
- name: Restart Tomcat
  systemd:
    name: tomcat
    state: restarted
```

Step 4: Build the Maven Ansible Web Application

In this step, we run the command mvn clean install to clean previous files and build the project. Maven compiles the application, packages it as a .war file, and stores it in the target folder. After successful completion, the build shows **BUILD SUCCESS** and installs the project files into the local Maven repository.

```
[INFO] No sources to compile
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MavenAnsibleWebApp1 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/NK/Devops/MavenAnsibleWebApp1/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ MavenAnsibleWebApp1 ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ MavenAnsibleWebApp1 ---
[INFO] No tests to run.
[INFO]
[INFO] --- maven-war-plugin:3.4.0:war (default-war) @ MavenAnsibleWebApp1 ---
[INFO] Packaging webapp
[INFO] Assembling webapp [MavenAnsibleWebApp1] in [/home/NK/Devops/MavenAnsibleWebApp1/target/MavenAnsibleWebApp1]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/NK/Devops/MavenAnsibleWebApp1/src/main/webapp]
[INFO] Building war: /home/NK/Devops/MavenAnsibleWebApp1/target/MavenAnsibleWebApp1.war
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MavenAnsibleWebApp1 ---
[INFO] Installing /home/NK/Devops/MavenAnsibleWebApp1/target/MavenAnsibleWebApp1.war to /home/NK/.m2/repository/com/example/MavenAnsibleWebAp
p1/1.0-SNAPSHOT/MavenAnsibleWebApp1-1.0-SNAPSHOT.war
[INFO] Installing /home/NK/Devops/MavenAnsibleWebApp1/pom.xml to /home/NK/.m2/repository/com/example/MavenAnsibleWebApp1/1.0-SNAPSHOT/MavenAn
sibleWebApp1-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.585 s
[INFO] Finished at: 2026-04-15T04:06:54Z
[INFO] ------------------------------------------------------------------------
```

Step 5: Verify Project Structure

In this step, we run the tree command to display the complete project structure. It shows important files and folders such as pom.xml, Jenkinsfile, ansible configuration files, source code folders, and the target directory containing the generated MavenAnsibleWebApp1.war file ready for deployment.

```
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp1$tree
.
├── ansible
│   ├── hosts.ini
│   └── playbook.yml
├── Jenkinsfile
├── pom.xml
├── src
│   └── main
│       ├── resources
│       └── webapp
│           ├── index.jsp
│           └── WEB-INF
│               └── web.xml
└── target
    ├── classes
    ├── MavenAnsibleWebApp1
    │   ├── index.jsp
    │   ├── META-INF
    │   └── WEB-INF
    │       ├── classes
    │       └── web.xml
    ├── MavenAnsibleWebApp..war
    └── maven-archiver
        └── pom.properties

14 directories, 10 files
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp $ █
```

Setting Up Jenkins Pipeline for MavenAnsibleWebApp

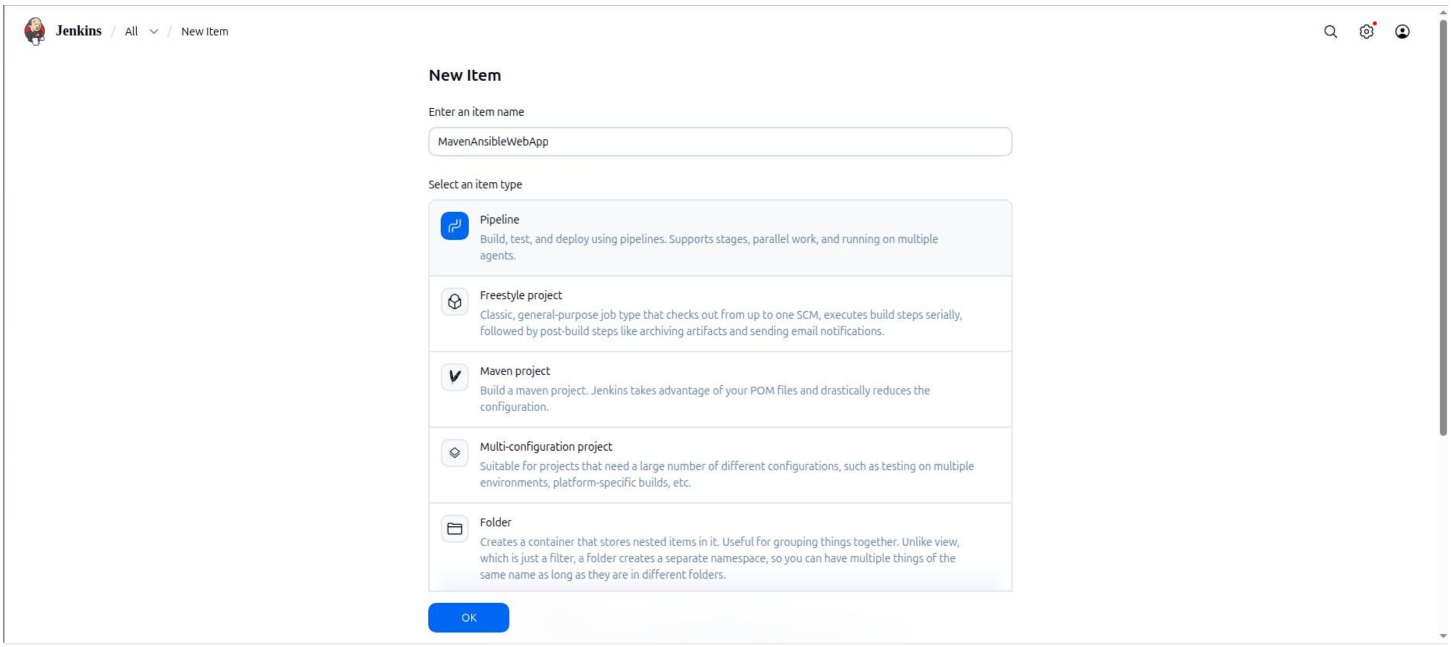
Step 1: To deploy **MavenAnsibleWebApp**, open the terminal, cd into the **MavenAnsibleWebApp** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

```
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp$ gedit Jenkinsfile
shweta@shweta-virtualbox:~/Devops/MavenAnsibleWebApp$ ls
Jenkinsfile  pom.xml  src
```

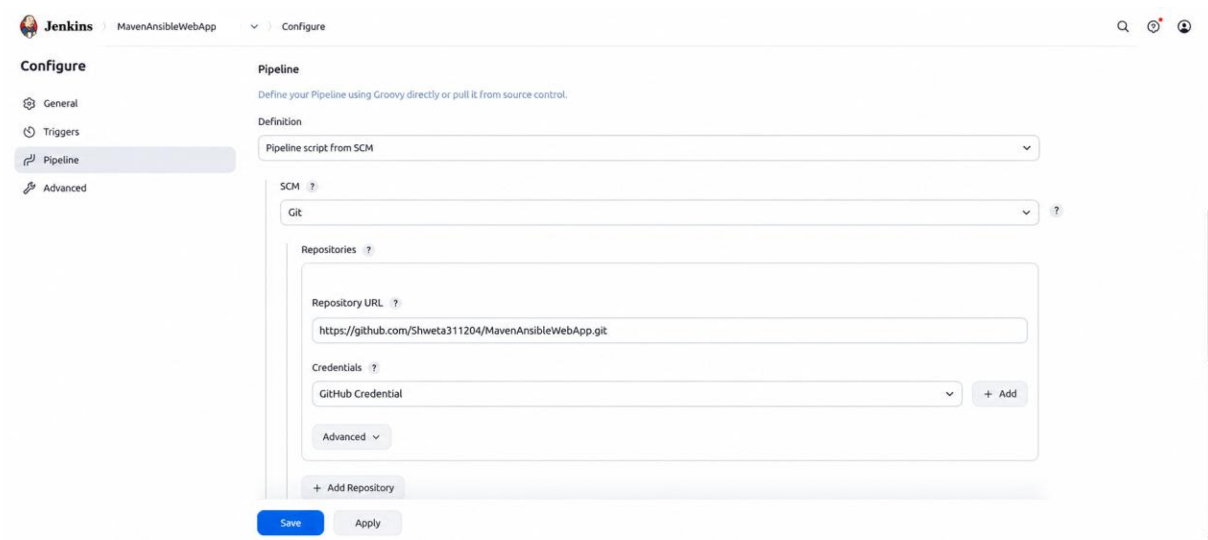
```
pipeline {
    agent any  // Use any available agent
    environment {
        LANG = 'en_US.UTF-8'
        LC_ALL = 'en_US.UTF-8'
    }    // this has to be added only if you get an error saying UTF required is 8 but showing in ISO00009
    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/Shweta311204/MavenAnsibleWebApp-.git'
            }
        }
```

```
    stage('Build') {
        steps {
            sh 'mvn clean package'  // Run Maven build
        }
    }
  stage('Archive') {
        steps {
            archiveArtifacts artifacts: 'target/*.war', fingerprint:true
        }
    }
    stage('Deploy') {
        steps {
           sh 'mvn clean package'
           sh 'ansible-playbook ansible/playbook.yml -i ansible/hosts.ini'
        }
    }
 }
}
```

Step 2: To deploy **MavenAnsibleWebApp**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on **"New Item"** to create a job, enter the item name (e.g., *MavenAnsibleWebApp*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click **"OK"** to proceed to the job configuration page.

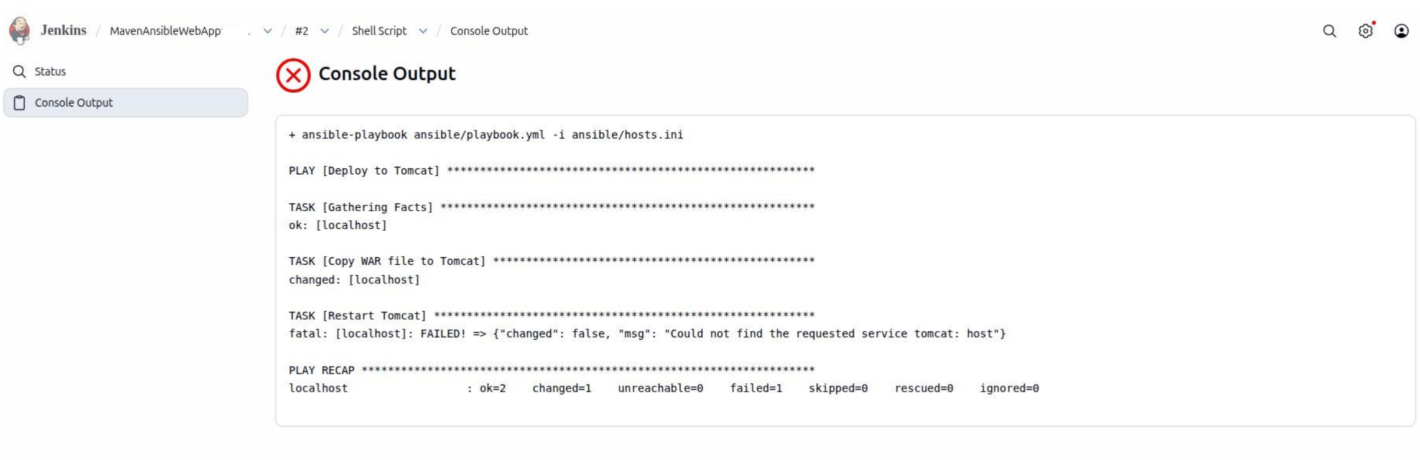


Step 3: In **MavenAnsibleWebApp**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MavenAnsibleWebApp**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MavenAnsibleWebApp folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: Build and Troubleshoot Deployment in Jenkins

After clicking **Apply and Save**, go back to MavenAnsibleWebApp in Jenkins and click **Build Now**. During the initial build, the deployment failed while restarting the Tomcat service because Ansible could not find the service name tomcat.



To resolve this issue, the playbook.yml file was updated by replacing the systemd restart method with Tomcat manual control commands. Instead of restarting a service named tomcat, the script uses /opt/tomcat/bin/shutdown.sh to stop the Tomcat server, waits for a few seconds using sleep 5, and then starts it again using /opt/tomcat/bin/startup.sh. This method works because Tomcat was installed manually and not registered as a system service. After saving the updated playbook, the Jenkins job was executed again, and the application was deployed successfully without any errors.

The below is the updated playbook.yml file, modified to resolve the Tomcat restart issue by using shutdown and startup commands instead of the systemd service method.

```
- name: Deploy to Tomcat
  hosts: web
  become: yes
  tasks:
    - name: Copy WAR file to Tomcat
      copy:
      src:
      "/var/snap/jenkins/5013/workspace/MavenAnsibleWebApp/target/MavenAnsibleWebA
      pp1.war"
        dest: "/opt/tomcat/webapps/MavenAnsibleWebApp1.war"
        remote_src: no
    - name: Restart Tomcat
      shell: |
        /opt/tomcat/bin/shutdown.sh || true
        sleep 5
        /opt/tomcat/bin/startup.sh
```

Step 5: After making the changes, go back to **MavenAnsibleWebApp1-CICD** in Jenkins and click **Build Now**. The build should complete successfully without any errors.

Verify MavenAnsibleWebApp Deployment on Tomcat

After completing the Tomcat installation and configuration, we open the terminal and switch to root user mode using sudo -i. Then, we move to the Tomcat binary directory using cd /opt/tomcat/bin and start the server using sh startup.sh. After Tomcat starts successfully, we open the browser and access http://localhost:9090/MavenAnsibleWebApp1 to verify the deployed application output.